

Project Overview

I implemented chords, dynamics, and echo. I use button 3 to toggle between states. Each press of the button increments a 2 bit counter by 1. Whenever the count is 0 or 2, the audio will sound normal. Whenever the count is 1, dynamics will be activated. Whenever the count is 3, echo will be activated.

Chords

Points: 4

Specification: This extension allows for playing one, two, or three notes at the same time. I altered the song rom so that the entries are 7 bit. The most significant bit encodes whether it is a note or a play command. The least significant 6 bits are the note or duration, depending on the first bit. This design modification decreases the amount of memory needed for storing songs.

High-level design: I implemented chords using an FSM with the following states:

1. WAIT_NEW_SONG (waits while there is no song to play)
2. ADVANCE_INDEX (advances the index to deal with the song rom's one cycle delay)
3. READ_NOTE_1 (stores note 1)
4. READ_NOTE_2 (stores note 2 if there is a second note)
5. READ_NOTE_3 (stores note 3 if there is a third note)
6. LATCH_CHORD (outputs a pulse, letting the note players know to start playing)
7. ADVANCE_SONG (waits for notes to finish playing)
8. SONG_FINISHED (song is finished playing)

This module feeds the respective notes to three instantiations of note players, which output three different samples. The number of notes is also tracked and used as the select signal for a mux. If there is one note, we select the first sample. If there are two notes, we average the first two notes. If there are three notes, we average all three notes. However, rather than divide by 3, we divide each of the samples by 4, which is easier to implement on hardware. The division occurs before the addition of samples to avoid clipping.

Key Implementation Details: A clever design technique I used was to connect the output of the song rom to the d ports of three flip flops, corresponding to each note. My FSM regulates the enable ports of all the flip flops. For example, in the state READ_NOTE_1, enable_1 will go high, which stores the first note in the first flip flop, without altering the information stored by the other flip flops.

Problems: The largest problem I encountered was not using signed variables for the samples. I was only able to resolve this problem in office hours.

Dynamics

Points: 4

Specification: This module adjusts the amplitude of the wave according to the ADSR envelope. The attack stage linearly increases the amplitude from 0 to 1. The decay state linearly decreases the amplitude from 1 to 0.5. The sustain stage keeps the amplitude at 0.5. The decay stage exponentially decreases the amplitude from 0.5 to 0.

High-level design: The amplitude is measured with a 5 bit number with a maximum value of 16, corresponding to an amplitude of 1.

I implemented dynamics using an FSM with the following states:

1. WAITING (Waits while there is no chord to play.)
2. ATTACK (Increases the amplitude by 2 for 8 beats.)
3. DECAY (Decreases the amplitude by 2 for 4 beats.)
4. SUSTAIN (Keeps the amplitude constant.)
5. RELEASE (Divides the amplitude by 2 every other beat for 10 beats.)

The duration of the sustain stage is the chord duration minus 22. Thus, for my implementation, the minimum duration is 22.

To compute the dynamic sample, the normal sample is first right shifted 4 times and then multiplied by the amplitude.

Key Implementation Details: A clever design technique I used was using the right shift operator to obtain exponential decay. At first, I was right shifting the amplitude every beat, which was too fast. To overcome this, I only right shifted the amplitude whenever the counter was even, which leads to a right shift every other beat.

Problems: The largest problem I encounter was using the multiplication operator. The amplitude variable had to be signed in order for the multiplication to work. I figure this out by doing research.

Echo

Points: 2

Specification: This module creates an echo in the audio by adding a delayed and attenuated audio.

High-level design: I implemented echo using an FSM with the following states:

1. WAITING (Waits while there is no chord to play.)
2. DELAY. (Counts the number of beats, ending at the desired delay).
3. LATCH (Tells the three delayed note players to start playing the current notes.)
4. ADVANCE (Waits for the current chord (not delayed) to finished playing.)

The delay is measure in beats. My implementation has a delay of 20 beats, equivalent to 20/48 seconds.

Key Implementation Details: A key insight I had was that I only needed to keep track of when new chords begin playing. This triggers the start of the delay stage. Whenever the desired delay is met, the current notes, duration, and number of notes are stored, and the three delayed note players are fed that information and told to start playing. The samples generated by the delayed note players are combined in the exact same way as chords to compute the echo. The echo is right shifted twice and the normal summed sample is right shifted once. These values are added to form the total echo sample.

Problems: The largest problem I encountered was not attenuation the original sample when adding it to the attenuated echo. This was causing clipping. To fix this, I right shifted the original sample once.